ChatGPT

Planning and Self-Reflection in Large Language Model Reasoning

Abstract

Planning and self-reflection have become two of the most important control principles for improving large language model reasoning. In the literature, **planning** refers to mechanisms that structure future reasoning or action before a final answer is produced, including decomposition, intermediate representations, search over candidate thoughts, and interaction with tools or environments. **Self-reflection** refers to mechanisms that monitor, critique, verify, and revise an already generated or partially generated reasoning trace, either through intrinsic language-only feedback or through extrinsic signals such as tests, retrieval, execution results, rule checkers, and process reward models. This survey argues that the two themes are best understood not as isolated prompting tricks, but as complementary strategies for **inference-time control over reasoning trajectories**. Planning broadens or organizes the trajectory space; reflection filters, repairs, or reallocates computation within that space. Across prompting, agentic control, process supervision, verifier training, and reinforcement learning, the field has moved from linear chain-of-thought elicitation toward increasingly explicit forms of search, monitoring, and meta-reasoning. At the same time, the strongest empirical results still depend heavily on reliable external feedback, carefully engineered intermediate representations, and evaluation domains that are more verifiable than open-ended natural language reasoning. The literature therefore suggests a nuanced conclusion: LLMs can often reason better when they plan and reflect, but robust gains usually arise when these capabilities are grounded by external structure rather than by unconstrained introspection alone [1–18, 25–31, 39–54, 56–70, 96–107]. 1

Introduction

The modern literature on LLM reasoning began by showing that sufficiently large models can be induced to generate intermediate steps through chain-of-thought prompting, zero-shot reasoning cues, and rationale sampling, which markedly improved arithmetic, symbolic, and commonsense reasoning performance [1–4, 22, 24]. From that point onward, two research questions quickly emerged. First, how can a model be encouraged to **plan** more effectively before committing to a final answer or action? Second, how can a model be induced to **reflect** on its own candidate reasoning and correct errors after the fact? These questions now sit at the core of work on reasoning models, language agents, process supervision, and inference-time scaling [5–13, 26–31, 39–54, 99–104]. 2

Although the two themes are tightly coupled, they are analytically distinct. Planning is primarily **prospective**: it selects subgoals, representations, or search policies before the full solution is known. Reflection is primarily **retrospective or interleaved**: it critiques a draft, verifies steps, or decides whether additional computation is warranted. In practice, many influential methods are hybrids. ReAct interleaves reasoning and acting [7]; RAP and LATS use planning algorithms to explore future reasoning states [5, 59, 107]; Chain-of-Verification, CRITIC, and Self-Debug turn verification into a structured refinement stage [27, 30, 31]; process reward model methods blur the boundary between reflection and search by scoring partial trajectories step by step [39–47]. The result is an increasingly unified view in which LLM reasoning is not just generation, but **controlled trajectory construction** under limited compute and imperfect feedback [44–47, 99, 101, 104, 106, 107]. 3

This survey focuses specifically on that controlled-trajectory perspective. It does not attempt to review all of LLM reasoning, nor all of agentic AI. Instead, it concentrates on methods that make planning or self-reflection explicit, the benchmark ecosystems that measure them, and the methodological tensions that continue to shape the field. Three recurrent conclusions appear throughout the literature. First, planning and reflection often help because they externalize latent computation into inspectable intermediate forms [1, 8, 9, 25, 96, 104]. Second, self-correction is much more reliable when it is anchored to extrinsic signals than when it is driven purely by free-form self-critique [26–33, 39–49, 68–70]. Third, long-horizon planning remains notably harder than short-horizon reasoning, even for strong frontier systems, particularly when tasks require persistent memory, tool orchestration, or adherence to many interacting constraints [62–67, 71–95, 101, 102]. 4

Methods

Planning-Centric Methods

The earliest planning-oriented methods largely **linearized** reasoning. Chain-of-thought, zero-shot chain-of-thought, least-to-most prompting, and plan-and-solve prompting all assume that difficult tasks become tractable when an LLM first decomposes them into smaller textual steps [1–6]. Closely related work explored modular prompting, abstraction prompts, analogy induction, and automatically composed reasoning structures, while scratchpad and program-aided methods shifted some intermediate computation into more constrained symbolic forms [11–18, 22–24]. This family established several durable lessons. Intermediate reasoning is often useful even when the final answer alone is evaluated; decomposition quality matters more than mere response length; and delegating arithmetic or symbolic execution to code interpreters can sharply reduce local error accumulation [4, 6, 12, 16, 17, 24, 96]. Yet these methods remain mostly **serial**: they improve reasoning by choosing a better path, not by explicitly searching many paths [1–4, 6, 12, 16, 17]. 5

A second family made planning more explicit by introducing **search over thoughts**. Tree of Thoughts generalized chain-of-thought from a single linear rationale to a branching search over coherent “thought” units [8]. Graph of Thoughts further generalized the dependency structure to arbitrary graphs [9], while Algorithm of Thoughts attempted to compress search-like control into algorithmic prompt patterns that require fewer model calls [10]. RAP brought Monte Carlo Tree Search into reasoning by treating the model as both a reasoning policy and a world model over intermediate states [5], and Stream of Search represented search traces directly in language for training and decoding [107]. LATS extended these ideas to language agents by integrating value estimates, environment feedback, and MCTS-like exploration into action selection [59]. Across these methods, planning is no longer just “think step by step”; it becomes **deliberate exploration under branching uncertainty** [5, 8–10, 59, 104, 107]. The major trade-off is cost. Branching search usually improves robustness only when the node evaluation signal is informative enough to justify the extra compute, and this remains highly task-dependent [8–10, 44–46, 104, 107]. 6

A third line of work grounds planning in **tools, environments, and embodied contexts** rather than in free-form text alone. ReAct showed that interleaving reasoning with actions can reduce hallucination and improve interactive decision-making [7]. In robotics and embodied settings, SayCan, Inner Monologue, ProgPrompt, and LLM-Planner demonstrated that grounding candidate plans in affordances, feedback, executable action grammars, and observed state can substantially improve reliability [55–58]. Voyager extended this pattern into lifelong open-ended skill acquisition [61]. Cognitive Architectures for Language Agents reframed such systems more abstractly as combinations of memory, action selection, and control modules [60]. The central finding in this literature is not that LLMs suddenly become classical planners, but that **textual world knowledge becomes more useful when constrained by environment dynamics and executable interfaces** [55–61]. This insight later shaped more realistic planning

benchmarks such as TravelPlanner, PARTNR, and long-horizon tool-use environments, where failures often arise not from local syntax but from constraint tracking, state maintenance, or strategic coordination [64, 71, 72, 92–95]. [7]

A fourth family shifts from prompt design to **training for search-aware reasoning**. STaR bootstrapped a model on its own successful rationales [19], while Quiet-STaR and Fast Quiet-STaR pushed reasoning inward, training models to generate or compress internal thought-like traces during prediction [20, 21]. A separate recent strand uses reinforcement learning and process-shaped objectives to induce longer and more structured reasoning, as in rStar-Math, PRIME, and SimpleRL-Zoo [50–52]. These methods suggest that planning is not only an inference-time scaffold; it can also become a learned behavior. However, the planning actually acquired is often domain-specific, especially in mathematics and code, and the literature still lacks strong evidence that the resulting policies transfer broadly across task families without extensive reward engineering or synthetic curriculum design [19–21, 44–52, 100, 103]. [8]

The evaluation ecosystem mirrors these methodological shifts. Early reasoning work concentrated on GSM8K, MATH, ARC, StrategyQA, BBH, and MMLU [40, 73–78]. As planning and agency became more prominent, benchmarks expanded to HotpotQA, FEVER, ALFWorld, WebShop, HumanEval, MBPP, CRUXEval, GPQA, LiveCodeBench, ProcessBench, PlanBench, TravelPlanner, PARTNR, BrowseComp, SPIN-Bench, DeepPlanning, and HeroBench [63, 64, 71–95]. What these benchmarks collectively reveal is that short-form answer accuracy and long-horizon planning competence are only weakly aligned. Models that perform strongly on arithmetic or multiple-choice reasoning may still fail badly when they must preserve global consistency across extended action sequences, gather information adaptively, or recover from earlier choices [62–67, 71–95]. [9]

Reflection-Centric and Hybrid Methods

Reflection-centric work initially treated self-correction as a **language-only post-processing loop**. Self-Refine generates feedback and then refines an answer using the same model [28]. Reflexion stores verbal lessons from prior failures and reuses them in subsequent attempts [29]. Self-Contrast improves feedback quality by contrasting inconsistent perspectives before revision [34]. In parallel, several studies directly tested whether naïve verbal reflection improves downstream accuracy and found mixed results: some improvements on structured tasks, but substantial sensitivity to prompt wording and task design [32–34, 68–70]. The broad lesson is that recognizing an error is not automatically easier than avoiding one. Reflection can work, but only under stricter conditions than many early papers implied [28, 29, 32–34, 68–70, 103]. [10]

A more reliable branch of the literature introduces **external or semi-external feedback** into reflection. Self-Verification checks a candidate answer by conditioning on it and reasoning backwards [26]. Chain-of-Verification plans explicit verification questions and answers them independently before producing a corrected response [27]. CRITIC uses tools to critique factual, numerical, or programmatic outputs [30]. Self-Debug leverages execution feedback or explanatory reanalysis to repair code [31]. In these methods, reflection is less like free-form introspection and more like **structured diagnosis**. The model is asked not simply to “think again,” but to run a verification protocol against intermediate artifacts. This shift from unconstrained self-talk to explicit checking procedures is one of the clearest advances in the literature [26, 27, 30, 31, 53]. [11]

The strongest reflection results increasingly come from **process supervision and verifier-guided selection** rather than from prompted reflection alone. Training verifiers for GSM8K already showed that rating multiple candidate solutions can outperform direct generation [40]. Let’s Verify Step by Step, Math-Shepherd, Generative Verifiers, ThinkPRM, and related work moved this idea from outcome supervision to step-level process supervision [39–43]. Later work then used self-verification or

reinforcement learning to internalize parts of this reflective function inside the generator itself [47–49]. This family is important because it reframes reflection as a **credit assignment problem over trajectories**. Instead of asking whether a model can write a good critique in natural language, it asks whether the system can assign useful preference mass to better intermediate traces. That perspective also explains why test-time scaling methods often depend on verifier quality: search helps only if some mechanism can distinguish promising trajectories from merely verbose ones [39–49]. [12]

Multi-agent reflection pushes the same intuition into **socially distributed critique**. Multiagent Debate, ReConcile, and ChatEval all replace self-critique by interaction among several model instances with different proposals, confidences, or roles [35–37]. In some settings this improves factuality, reasoning, or evaluation quality [35–37]. But later analyses found that a strong single-agent prompt can sometimes match or nearly match the best group-discussion setups, casting doubt on whether debate helps because of genuine epistemic diversity or merely because it expands the sample budget [38]. This is an important methodological caution: multi-agent reflection should be compared not only with weak greedy decoding, but also with carefully matched single-agent baselines that receive comparable compute [35–38, 44, 45]. [13]

The newest strand of reflection research is explicitly **metacognitive**. Recent proposals ground reflection in psychologically inspired regulatory loops of planning, monitoring, and evaluation, and then add mechanisms for accumulating reusable meta-level knowledge across episodes [53, 54]. In parallel, surveys of self-evolution frame self-improvement as an iterative cycle of experience generation, refinement, updating, and evaluation [103]. These works are still early, and many remain in preprint form, but they indicate a conceptual shift. Reflection is no longer viewed only as a single repair pass; it is increasingly treated as a persistent control process that can learn **when to reflect, what to check, and how much extra computation to spend** [32, 44–54, 103]. [14]

Challenges and Prospects

A first persistent challenge is **faithfulness**. A reasoning trace that improves answer accuracy is not necessarily a transparent account of the causal basis of the answer. CoT can remain useful even with flawed demonstrations [96], and generated explanations can rationalize biased or incorrect outputs without revealing the true decision basis [97]. More recent analysis continues to question whether intermediate traces in reasoning models should be anthropomorphized as direct windows into thought [105, 106]. For planning and reflection research, this matters because many methods assume that better intermediate text implies better internal control. The empirical record is more cautious: textually elaborate reasoning can correlate with improved performance, but it is neither a guaranteed explanation nor a guaranteed verifier [25, 33, 96–98, 105, 106]. [15]

A second challenge is that **self-correction remains brittle without trustworthy feedback**. The critical survey by Kamoi et al. argues that successful self-correction has rarely been demonstrated under purely prompted, language-only feedback except in unusually favorable settings [32]. Prompt wording alone can flip reflective outcomes [33]. Studies on planning and graph coloring also show that iterative critique often fails when the critique source is itself unreliable [68, 69]. CorrectBench further suggests that even by 2025 the gains from self-correction vary strongly by task, method family, and efficiency budget [70]. The cumulative evidence points toward a simple but important principle: reflection is not a magic second chance; it is another inference problem, and it fails when the checking channel is as noisy as the generation channel [26, 27, 30–33, 39–49, 68–70]. [16]

A third challenge is **long-horizon planning under realistic constraints**. Benchmarks such as PlanBench, TravelPlanner, SPIN-Bench, DeepPlanning, and HeroBench consistently show that constraint coupling,

dynamic information gathering, and path dependence remain hard [63, 64, 92–95]. The literature therefore supports the increasingly common "LLM-modulo" view: the most reliable systems often use LLMs as proposal generators, translators, critics, or heuristics inside larger systems that also include sound planners, simulators, retrievers, interpreters, or rule-based validators [5, 16, 17, 39–47, 55–67]. This is not a weakness of LLMs per se; rather, it reflects the mismatch between unconstrained text prediction and the discrete, globally coupled structure of many planning problems. The research opportunity is to design interfaces that preserve the expressive advantages of language while offloading exact reasoning to external modules whenever verifiability or safety demands it [55–67, 101, 102]. [17]

The most plausible prospects lie in **unified controllers for planning, verification, and compute allocation**. Test-time scaling work suggests that additional computation is valuable only when allocated selectively and paired with strong verifiers [44–47]. Planning surveys increasingly organize methods around external modules, search, and fine-tuning rather than around prompting alone [101, 102]. ProcessBench and related datasets indicate that step-level error localization may be a major bottleneck for future scalable oversight [90]. A likely next phase, then, is not a single dominant "better prompt," but architectures that learn when to branch, when to execute tools, when to verify, when to revise, and when to stop. In other words, the field is moving from "reasoning traces" toward **reasoning control policies** [43–54, 90, 101–107]. [18]

Conclusion

The literature on planning and self-reflection in LLM reasoning has matured from a collection of prompting heuristics into a broader study of inference-time control. Planning methods improve reasoning by reshaping the trajectory space through decomposition, intermediate programs, search, and environment-grounded action selection. Reflection methods improve reasoning by diagnosing, scoring, repairing, or rejecting those trajectories. The most convincing results arise when the two are combined: planned exploration creates candidate structure, and reflective verification selects or repairs that structure. Yet the field's strongest empirical pattern is also its main caveat. LLMs become much more reliable reasoners when paired with **externalized structure**—interpreters, search procedures, process labels, environment state, retrieval, or formal validators. Purely intrinsic self-reflection remains far less dependable [1–17, 26–32, 39–49, 55–67, 99–106]. [19]

For that reason, the central question for future work is not simply whether LLMs can "think better," but how systems should distribute reasoning labor across language models, search, memory, tools, and verifiers. Planning and self-reflection have already shown that LLM reasoning is most powerful when it is not treated as a single left-to-right pass, but as a controlled process that can branch, inspect itself, and incorporate grounded feedback. The remaining challenge is to make that control efficient, faithful, and robust across the less forgiving domains where open-ended language, strategic interaction, and long-horizon constraints meet. [20]

References

[1] Jason Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," NeurIPS, 2022. arXiv:2201.11903.

[2] Takeshi Kojima et al., "Large Language Models are Zero-Shot Reasoners," NeurIPS, 2022. arXiv: 2205.11916.

[3] Xuezhi Wang et al., "Self-Consistency Improves Chain of Thought Reasoning in Language Models," ICLR, 2023. arXiv:2203.11171.

[4] Denny Zhou et al., “Least-to-Most Prompting Enables Complex Reasoning in Large Language Models,” ICLR, 2023. arXiv:2205.10625.

[5] Shibo Hao et al., “Reasoning with Language Model is Planning with World Model,” EMNLP, 2023. arXiv:2305.14992.

[6] Lei Wang et al., “Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models,” ACL, 2023. arXiv:2305.04091.

[7] Shunyu Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” ICLR, 2023. arXiv: 2210.03629.

[8] Shunyu Yao et al., “Tree of Thoughts: Deliberate Problem Solving with Large Language Models,” NeurIPS, 2023. arXiv:2305.10601.

[9] Maciej Besta et al., “Graph of Thoughts: Solving Elaborate Problems with Large Language Models,” AAAI, 2024. arXiv:2308.09687.

[10] Bilgehan Sel et al., “Algorithm of Thoughts: Enhancing Exploration of Ideas in Large Language Models,” AAAI, 2024. arXiv:2308.10379.

[11] Pei Zhou et al., “Self-Discover: Large Language Models Self-Compose Reasoning Structures,” NeurIPS, 2024. arXiv:2402.03620.

[12] Huaixiu Steven Zheng et al., “Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models,” ICLR, 2024. arXiv:2310.06117.

[13] Chengrun Yang et al., “Large Language Models as Optimizers,” ICLR, 2024. arXiv:2309.03409.

[14] Antonia Creswell, Murray Shanahan, and Irina Higgins, “Selection-Inference: Exploiting Large Language Models for Interpretable Logical Reasoning,” arXiv preprint, 2022. arXiv:2205.09712.

[15] Tushar Khot et al., “Decomposed Prompting: A Modular Approach for Solving Complex Tasks,” ICLR, 2023. arXiv:2210.02406.

[16] Luyu Gao et al., “PAL: Program-Aided Language Models,” ICML, 2023. arXiv:2211.10435.

[17] Wenhu Chen et al., “Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks,” Transactions on Machine Learning Research, 2023. arXiv:2211.12588.

[18] Maxwell Nye et al., “Show Your Work: Scratchpads for Intermediate Computation with Language Models,” Deep Learning for Code Workshop at ICLR, 2022. arXiv:2112.00114.

[19] Eric Zelikman et al., “STaR: Bootstrapping Reasoning With Reasoning,” NeurIPS, 2022. arXiv: 2203.14465.

[20] Eric Zelikman et al., “Quiet-STaR: Language Models Can Teach Themselves to Think Before Speaking,” arXiv preprint, 2024. arXiv:2403.09629.

[21] Wei Huang et al., “Fast Quiet-STaR: Thinking Without Thought Tokens,” Findings of EMNLP, 2025. arXiv:2505.17746.

[22] Xuezhi Wang et al., “Rationale-Augmented Ensembles in Language Models,” arXiv preprint, 2022. arXiv:2207.00747.

[23] Michihiro Yasunaga et al., “Large Language Models as Analogical Reasoners,” arXiv preprint, 2023. arXiv:2310.01714.

[24] Aman Madaan and Amir Yazdanbakhsh, “Text and Patterns: For Effective Chain of Thought, It Takes Two to Tango,” arXiv preprint, 2022. arXiv:2209.07686.

[25] Antonia Creswell and Murray Shanahan, “Faithful Reasoning Using Large Language Models,” arXiv preprint, 2022. arXiv:2208.14271.

[26] Yixuan Weng et al., “Large Language Models are Better Reasoners with Self-Verification,” Findings of EMNLP, 2023. arXiv:2212.09561.

[27] Shehzaad Dhuliawala et al., “Chain-of-Verification Reduces Hallucination in Large Language Models,” ACL, 2024. arXiv:2309.11495.

[28] Aman Madaan et al., “Self-Refine: Iterative Refinement with Self-Feedback,” NeurIPS, 2023. arXiv: 2303.17651.

[29] Noah Shinn et al., “Reflexion: Language Agents with Verbal Reinforcement Learning,” NeurIPS, 2024. arXiv:2303.11366.

[30] Zhibin Gou et al., “CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing,” ICLR, 2024. arXiv:2305.11738.

[31] Xinyun Chen et al., “Teaching Large Language Models to Self-Debug,” ICLR, 2024. arXiv:2304.05128.

[32] Ryo Kamoi et al., “When Can LLMs Actually Correct Their Own Mistakes? A Critical Survey of Self-Correction of LLMs,” Transactions of the Association for Computational Linguistics, 2024. arXiv: 2406.01297.

[33] Fengyuan Liu et al., “Self-Reflection Outcome is Sensitive to Prompt Construction,” arXiv preprint, 2024. arXiv:2406.10400.

[34] Wenqi Zhang et al., “Self-Contrast: Better Reflection Through Inconsistent Solving Perspectives,” ACL, 2024. arXiv:2401.02009.

[35] Yilun Du et al., “Improving Factuality and Reasoning in Language Models through Multiagent Debate,” ICML, 2024. arXiv:2305.14325.

[36] Justin Chih-Yao Chen, Swarnadeep Saha, and Mohit Bansal, “ReConcile: Round-Table Conference Improves Reasoning via Consensus among Diverse LLMs,” ACL, 2024. arXiv:2309.13007.

[37] Chi-Min Chan et al., “ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate,” ICLR, 2024. arXiv:2308.07201.

[38] Qineng Wang et al., “Rethinking the Bounds of LLM Reasoning: Are Multi-Agent Discussions the Key?,” ACL, 2024. arXiv:2402.18272.

[39] Hunter Lightman et al., “Let’s Verify Step by Step,” ICLR, 2024. arXiv:2305.20050.

[40] Karl Cobbe et al., “Training Verifiers to Solve Math Word Problems,” arXiv preprint, 2021. arXiv: 2110.14168.

[41] Peiyi Wang et al., “Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations,” ACL, 2024. arXiv:2312.08935.

[42] Lunjun Zhang et al., “Generative Verifiers: Reward Modeling as Next-Token Prediction,” arXiv preprint, 2024. arXiv:2408.15240.

[43] Muhammad Khalifa et al., “Process Reward Models That Think,” Transactions on Machine Learning Research, 2025. arXiv:2504.16828.

[44] Charlie Snell et al., “Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters,” arXiv preprint, 2024. arXiv:2408.03314.

[45] Vidhisha Balachandran et al., “Inference-Time Scaling for Complex Tasks: Where We Stand and What Lies Ahead,” arXiv preprint, 2025. arXiv:2504.00294.

[46] Amrith Setlur et al., “Scaling Test-Time Compute Without Verification or RL is Suboptimal,” arXiv preprint, 2025. arXiv:2502.12118.

[47] Fuxiang Zhang et al., “Incentivizing LLMs to Self-Verify Their Answers,” arXiv preprint, 2025. arXiv: 2506.01369.

[48] Shelly Bensal et al., “Reflect, Retry, Reward: Self-Improving LLMs via Reinforcement Learning,” arXiv preprint, 2025. arXiv:2505.24726.

[49] Renze Ma et al., “S2R: Teaching LLMs to Self-verify and Self-correct via Reinforcement Learning,” ACL, 2025. arXiv:2502.12853.

[50] Xinyu Guan et al., “rStar-Math: Small LLMs Can Master Math Reasoning with Self-Evolved Deep Thinking,” arXiv preprint, 2025. arXiv:2501.04519.

[51] Guangyuan Cui et al., “Process Reinforcement through Implicit Rewards,” ICLR, 2025. arXiv: 2502.01456.

[52] Weihao Zeng et al., “SimpleRL-Zoo: Investigating and Taming Zero Reinforcement Learning for Open Base Models in the Wild,” arXiv preprint, 2025. arXiv:2503.18892.

[53] Abraham Paul Elenjical, Vivek Hruday Kavuri, and Vasudeva Varma, “Think^2: Grounded Metacognitive Reasoning in Large Language Models,” arXiv preprint, 2026. arXiv:2602.18806.

[54] Ziqing Zhuang et al., “Beyond Meta-Reasoning: Metacognitive Consolidation for Self-Improving LLM Reasoning,” ACL, 2026. arXiv:2604.17399.

[55] Michael Ahn et al., “Do As I Can, Not As I Say: Grounding Language in Robotic Affordances,” Conference on Robot Learning, 2022. arXiv:2204.01691.

[56] Wenlong Huang et al., “Inner Monologue: Embodied Reasoning through Planning with Language Models,” Conference on Robot Learning, 2023. arXiv:2207.05608.

[57] Ishika Singh et al., “ProgPrompt: Generating Situated Robot Task Plans using Large Language Models,” ICRA, 2023. arXiv:2209.11302.

[58] Chan Hee Song et al., “LLM-Planner: Few-Shot Grounded Planning for Embodied Agents with Large Language Models,” ICCV, 2023. arXiv:2212.04088.

[59] Andy Zhou et al., “Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models,” ICML, 2024. arXiv:2310.04406.

[60] Theodore R. Sumers et al., “Cognitive Architectures for Language Agents,” Transactions on Machine Learning Research, 2024. arXiv:2309.02427.

[61] Guanzhi Wang et al., “Voyager: An Open-Ended Embodied Agent with Large Language Models,” arXiv preprint, 2023. arXiv:2305.16291.

[62] Karthik Valmeekam et al., “On the Planning Abilities of Large Language Models: A Critical Investigation,” NeurIPS, 2023. arXiv:2305.15771.

[63] Karthik Valmeekam et al., “PlanBench: An Extensible Benchmark for Evaluating Large Language Models on Planning and Reasoning about Change,” NeurIPS, 2023. arXiv:2206.10498.

[64] Jian Xie et al., “TravelPlanner: A Benchmark for Real-World Planning with Language Agents,” ICML, 2024. arXiv:2402.01622.

[65] Yaqi Xie et al., “Translating Natural Language to Planning Goals with Large-Language Models,” ICRA, 2024. arXiv:2302.05128.

[66] Subbarao Kambhampati et al., “Position: LLMs Can’t Plan, But Can Help Planning in LLM-Modulo Frameworks,” ICML, 2024.

[67] Subbarao Kambhampati, “Can Large Language Models Reason and Plan?,” Annals of the New York Academy of Sciences, 2024. arXiv:2403.04121.

[68] Karthik Valmeekam, Matthew Marquez, and Subbarao Kambhampati, “Can Large Language Models Really Improve by Self-critiquing Their Own Plans?,” arXiv preprint, 2023. arXiv:2310.08118.

[69] Kaya Stechly, Matthew Marquez, and Subbarao Kambhampati, “GPT-4 Doesn’t Know It’s Wrong: An Analysis of Iterative Prompting for Reasoning Problems,” arXiv preprint, 2023. arXiv:2310.12397.

[70] Guiyao Tie et al., “Can LLMs Correct Themselves? A Benchmark of Self-Correction in LLMs,” arXiv preprint, 2025. arXiv:2510.16062.

[71] Andrew Liu et al., “PARTNR: A Benchmark for Planning and Reasoning in Embodied Multi-Agent Tasks,” arXiv preprint, 2024. arXiv:2411.00081.

[72] Jiayu Liu et al., “PlanBench-XL: Evaluating Long-Horizon Planning of LLM Tool-Use Agents in Large-Scale Tool Ecosystems,” arXiv preprint, 2026. arXiv:2606.22388.

[73] Dan Hendrycks et al., “Measuring Mathematical Problem Solving With the MATH Dataset,” NeurIPS, 2021. arXiv:2103.03874.

[74] Dan Hendrycks et al., “Measuring Massive Multitask Language Understanding,” ICLR, 2021. arXiv: 2009.03300.

[75] BIG-bench Collaboration, “Beyond the Imitation Game: Quantifying and Extrapolating the Capabilities of Language Models,” Transactions on Machine Learning Research, 2023. arXiv:2206.04615.

[76] Mirac Suzgun et al., “Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them,” Findings of ACL, 2023. arXiv:2210.09261.

[77] Mor Geva et al., “Did Aristotle Use a Laptop? A Question Answering Benchmark with Implicit Reasoning Strategies,” Transactions of the Association for Computational Linguistics, 2021. arXiv: 2101.02235.

[78] Peter Clark et al., “Think You Have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge,” arXiv preprint, 2018. arXiv:1803.05457.

[79] Alon Talmor and Jonathan Berant, “The Web as a Knowledge-Base for Answering Complex Questions,” NAACL, 2018. arXiv:1803.06643.

[80] Adina Williams et al., “A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference,” NAACL, 2018. arXiv:1704.05426.

[81] Johannes Welbl, Pontus Stenetorp, and Sebastian Riedel, “Constructing Datasets for Multi-hop Reading Comprehension Across Documents,” Transactions of the Association for Computational Linguistics, 2018. arXiv:1710.06481.

[82] Neil Thorne et al., “FEVER: A Large-scale Dataset for Fact Extraction and Verification,” NAACL, 2018. arXiv:1803.05355.

[83] Shunyu Yao et al., “WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents,” NeurIPS, 2022. arXiv:2207.01206.

[84] Mohit Shridhar et al., “ALFWorld: Aligning Text and Embodied Environments for Interactive Learning,” ICLR, 2021. arXiv:2010.03768.

[85] Jacob Austin et al., “Program Synthesis with Large Language Models,” arXiv preprint, 2021. arXiv: 2108.07732.

[86] Mark Chen et al., “Evaluating Large Language Models Trained on Code,” arXiv preprint, 2021. arXiv: 2107.03374.

[87] Alex Gu et al., “CRUXEval: A Benchmark for Code Reasoning, Understanding and Execution,” arXiv preprint, 2024. arXiv:2401.03065.

[88] David Rein et al., “GPQA: A Graduate-Level Google-Proof Q&A Benchmark,” ICLR, 2024. arXiv: 2311.12022.

[89] Naman Jain et al., “LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code,” arXiv preprint, 2024. arXiv:2403.07974.

[90] Chujie Zheng et al., “ProcessBench: Identifying Process Errors in Mathematical Reasoning,” arXiv preprint, 2024. arXiv:2412.06559.

[91] Jason Wei et al., “BrowseComp: A Simple Yet Challenging Benchmark for Browsing Agents,” arXiv preprint, 2025. arXiv:2504.12516.

[92] Jianzhu Yao et al., “SPIN-Bench: How Well Do LLMs Plan Strategically and Reason Socially?,” arXiv preprint, 2025. arXiv:2503.12349.

[93] Yinger Zhang et al., “DeepPlanning: Benchmarking Long-Horizon Agentic Planning with Verifiable Constraints,” arXiv preprint, 2026. arXiv:2601.18137.

[94] Petr Anokhin et al., “HeroBench: A Benchmark for Long-Horizon Planning and Structured Reasoning in Virtual Worlds,” arXiv preprint, 2025. arXiv:2508.12782.

[95] Haoming Li et al., “A Collection of Benchmarks for Evaluating LLMs’ Planning Capability,” arXiv preprint, 2025. arXiv:2504.14773.

[96] Boshi Wang et al., “Towards Understanding Chain-of-Thought Prompting: An Empirical Study of What Matters,” ACL, 2023. arXiv:2212.10001.

[97] Miles Turpin et al., “Language Models Don’t Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting,” NeurIPS, 2023. arXiv:2305.04388.

[98] Ben Prystawski et al., “Why Think Step by Step? Reasoning Emerges from the Locality of Experience,” NeurIPS, 2023. arXiv:2304.03843.

[99] Aske Plaat et al., “Reasoning with Large Language Models, a Survey,” arXiv preprint, 2024. arXiv: 2407.11511.

[100] Janice Ahn et al., “Large Language Models for Mathematical Reasoning: Progresses and Challenges,” arXiv preprint, 2024. arXiv:2402.00157.

[101] Pengfei Cao et al., “Large Language Models for Planning: A Comprehensive and Systematic Survey,” arXiv preprint, 2025. arXiv:2505.19683.

[102] Hui Wei et al., “PlanGenLLMs: A Modern Survey of LLM Planning Capabilities,” ACL, 2025. arXiv: 2502.11221.

[103] Zhengwei Tao et al., “A Survey on Self-Evolution of Large Language Models,” arXiv preprint, 2024. arXiv:2404.14387.

[104] Maciej Besta et al., “Demystifying Chains, Trees, and Graphs of Thoughts,” IEEE Transactions on Pattern Analysis and Machine Intelligence, 2025. arXiv:2401.14295.

[105] Rui Yang et al., “Do Large Language Models Latently Perform Multi-Hop Reasoning?,” arXiv preprint, 2024. arXiv:2402.16837.

[106] Subbarao Kambhampati, Kaya Stechly, and Karthik Valmeekam, “How Do Reasoning Models Reason?,” Annals of the New York Academy of Sciences, 2025. arXiv:2504.09762.

[107] Kanishk Gandhi et al., “Stream of Search: Learning to Search in Language,” COLM, 2024. arXiv: 2404.03683.

---

1 2 5 Chain-of-Thought Prompting Elicits Reasoning in Large Language Models
https://arxiv.org/abs/2201.11903?utm_source=chatgpt.com

3 7 ReAct: Synergizing Reasoning and Acting in Language Models
https://arxiv.org/abs/2210.03629?utm_source=chatgpt.com

4 Can Large Language Models Reason and Plan?
https://arxiv.org/abs/2403.04121?utm_source=chatgpt.com

6 Tree of Thoughts: Deliberate Problem Solving with Large Language Models
https://arxiv.org/abs/2305.10601?utm_source=chatgpt.com

8 STaR: Bootstrapping Reasoning With Reasoning
https://arxiv.org/abs/2203.14465?utm_source=chatgpt.com

9 Training Verifiers to Solve Math Word Problems
https://arxiv.org/abs/2110.14168?utm_source=chatgpt.com

10 Self-Refine: Iterative Refinement with Self-Feedback
https://arxiv.org/abs/2303.17651?utm_source=chatgpt.com

11 Large Language Models are Better Reasoners with Self-Verification
https://arxiv.org/abs/2212.09561?utm_source=chatgpt.com

12 Let's Verify Step by Step
https://arxiv.org/abs/2305.20050?utm_source=chatgpt.com

13 Improving Factuality and Reasoning in Language Models through Multiagent Debate
https://arxiv.org/abs/2305.14325?utm_source=chatgpt.com

14 Think^{2}: Grounded Metacognitive Reasoning in Large Language Models
https://arxiv.org/abs/2602.18806?utm_source=chatgpt.com

15 Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting
https://arxiv.org/abs/2305.04388?utm_source=chatgpt.com

16 When Can LLMs Actually Correct Their Own Mistakes? A Critical Survey of Self-Correction of LLMs
https://arxiv.org/abs/2406.01297?utm_source=chatgpt.com

17 On the Planning Abilities of Large Language Models (A Critical Investigation with a Proposed Benchmark)
https://arxiv.org/abs/2302.06706?utm_source=chatgpt.com

18 Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters
https://arxiv.org/abs/2408.03314?utm_source=chatgpt.com

19 Reasoning with Large Language Models, a Survey
https://arxiv.org/abs/2407.11511?utm_source=chatgpt.com

20 Large Language Models for Planning: A Comprehensive and Systematic Survey
https://arxiv.org/abs/2505.19683?utm_source=chatgpt.com